Level 2
Hackathon – Fundamentals of Web Development

Student Details

Student Name	Dharshini.K
Register Number	34625U50007
Department	BSc (Artificial Intelligence)
ID	218ee7e1ae00952aa5e42b76875de1bf@vtpt2026.in
College	MMES Women's Arts and Science College
Date of Submission	28-10-2026

Git-Hub Link:
https://github.com/<your-username>/online-food-ordering-website

1. Problem Statement
Many small restaurants and food stalls still depend on phone calls or manual order-taking, which causes slow service, wrong orders, and confusion about prices. Customers also find it difficult to see the complete menu, compare items, and know the final bill before ordering. The problem is to develop an online food ordering website that lets customers browse food items, search and filter dishes, add products to a cart, manage quantities, and review their order before submission. This application is needed because it makes ordering faster, accurate, and convenient for both customers and restaurants.

2. Objectives
- Display food items dynamically with images, names, descriptions, categories and prices.
- Allow customers to filter food items by category and search them by name.
- Provide a shopping cart to add/remove items and increase/decrease quantities.
- Calculate item subtotals and the overall order total dynamically.
- Validate customer details and show an order summary before submission.
- Store cart data in Local Storage so it is restored when the page is reopened.
- Show appropriate messages when the cart is empty or an order is submitted.

Expected outcome: a responsive, easy-to-use website where a customer can place an order smoothly without page reloads and without losing cart data.

3. Architecture
The application is a single-page, client-side web application made of three files: **index.html** (structure), **style.css** (design) and **script.js** (logic). The page has a menu section (search box, category filter, food cards), a cart sidebar (items, quantities, total, customer form) and an order summary pop-up.

Technologies used and their roles:

- **HTML5** – builds the page structure: header, search box, dropdown, cart, form and modal.
- **CSS3** – styles the layout using Flexbox and Grid, cards, buttons and responsive media queries.
- **JavaScript (ES6)** – renders food items dynamically, handles events, cart logic, totals and validation.

- **Local Storage (Web Storage API)** – saves the cart in the browser and restores it on reopening.
- **VS Code / Browser / GitHub** – used for development, running and hosting the source code.

Application flow diagram:



4. Practical Implementation

The food items are stored as an array of JavaScript objects (id, name, category, price, emoji image, description). The page is built in HTML and styled with CSS. JavaScript then reads the array and creates a card for every item using the DOM. Event listeners on the search box and category dropdown re-render the list instantly. The cart is an array of {id, qty} objects; every change updates the screen and is saved to Local Storage.

4.1 Features Implemented

- **Dynamic food display** – shows image, name, description, category and price from data.
- **Category filter** – dropdown filters items by category.
- **Search by name** – live search as the user types.
- **Add / remove from cart** – 'Add to Cart' button and 'x' remove button.
- **Quantity control** – '+' and '-' buttons change the quantity of each item.
- **Dynamic totals** – subtotal per item and the grand total update automatically.
- **Order summary** – pop-up listing items, quantities, prices and customer details.
- **Form validation** – checks name, 10-digit phone number, address and non-empty cart.
- **Local Storage** – cart is restored when the page is reopened.
- **Messages** – 'Your cart is empty' and 'Order submitted successfully' messages.

4.2 Source Code

HTML

HTML defines the structure: the header with cart count, search input, category dropdown, food list container, cart sidebar with customer form, and the hidden order summary modal. IDs are used so JavaScript can access each element.

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>FoodieHub - Online Food Ordering</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>FoodieHub</h1>
    <span class="cart-badge">Cart: <b id="cartCount">0</b></span>
  </header>

  <main>
    <section class="menu">
      <div class="controls">
        <input type="text" id="searchInput" placeholder="Search food by name...">
        <select id="categoryFilter">
          <option value="All">All Categories</option>
        </select>
      </div>
      <div id="foodList" class="food-grid"></div>
    </section>

    <aside class="cart">
      <h2>Your Cart</h2>
      <div id="cartItems"></div>
      <p id="cartMessage" class="msg"></p>
      <h3>Total: &#8377;<span id="cartTotal">0</span></h3>

      <h2>Customer Details</h2>
      <input type="text" id="custName" placeholder="Full Name">
      <input type="text" id="custPhone" placeholder="Phone (10 digits)">
      <input type="text" id="custAddress" placeholder="Delivery Address">
      <p id="formError" class="error"></p>
      <button id="reviewBtn">Review Order</button>
    </aside>
  </main>
  <div id="summaryModal" class="modal hidden">
    <div class="modal-box">
      <h2>Order Summary</h2>
      <div id="summaryContent"></div>
      <button id="confirmBtn">Confirm Order</button>
      <button id="cancelBtn" class="secondary">Cancel</button>
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

CSS

CSS gives the website its look: orange theme, Flexbox layout for menu and cart, CSS Grid for food cards, styled buttons and inputs, a full-screen modal overlay, and a media query to stack the layout on small screens.

style.css

```
* { box-sizing: border-box; margin: 0; padding: 0; font-family: Arial, sans-serif; }
body { background: #fff7ee; color: #333; }
header { background: #e65100; color: #fff; padding: 14px 24px;
         display: flex; justify-content: space-between; align-items: center; }
.cart-badge { background: #fff; color: #e65100; padding: 6px 14px; border-radius: 20px; }
main { display: flex; gap: 20px; padding: 20px; flex-wrap: wrap; }
.menu { flex: 3; min-width: 300px; }
.cart { flex: 1; min-width: 280px; background: #fff; padding: 16px;
        border-radius: 10px; box-shadow: 0 2px 8px rgba(0,0,0,.15); }
.controls { display: flex; gap: 10px; margin-bottom: 16px; }
input, select { width: 100%; padding: 9px; margin: 6px 0;
                border: 1px solid #bbb; border-radius: 6px; }
.food-grid { display: grid; gap: 16px;
             grid-template-columns: repeat(auto-fill, minmax(190px, 1fr)); }
.card { background: #fff; border-radius: 10px; overflow: hidden;
        box-shadow: 0 2px 6px rgba(0,0,0,.15); }
.card .img { height: 110px; display: flex; align-items: center;
             justify-content: center; font-size: 56px; background: #ffe0b2; }
.card .body { padding: 10px; }
.card h4 { margin-bottom: 4px; }
.card p { font-size: 13px; color: #666; margin-bottom: 6px; }
.tag { font-size: 11px; background: #ffcc80; padding: 2px 8px; border-radius: 10px; }
.price { font-weight: bold; color: #e65100; margin: 6px 0; }
button { background: #e65100; color: #fff; border: none; padding: 9px 14px;
         border-radius: 6px; cursor: pointer; width: 100%; margin-top: 6px; }
button.secondary { background: #888; }
button:hover { opacity: .9; }
.cart-row { display: flex; justify-content: space-between; align-items: center;
            border-bottom: 1px solid #eee; padding: 8px 0; font-size: 14px; }
.qty button { width: 26px; padding: 2px; margin: 0 3px; }
.msg { color: #2e7d32; margin: 8px 0; font-weight: bold; }
.error { color: #c62828; font-size: 13px; }
.modal { position: fixed; inset: 0; background: rgba(0,0,0,.5);
         display: flex; align-items: center; justify-content: center; }
.modal.hidden { display: none; }
.modal-box { background: #fff; padding: 20px; border-radius: 10px;
             width: 90%; max-width: 420px; max-height: 90vh; overflow: auto; }
@media (max-width: 600px) { main { flex-direction: column; } }
```

JavaScript

JavaScript implements the logic: **renderFoods()** filters and displays items, **addToCart()**, **changeQty()** and **removeItem()** manage the cart, **renderCart()** calculates subtotals and total, **validateForm()** validates the details, and **saveCart()** stores the cart in Local Storage.

script.js

```
const foods = [
  { id: 1, name: "Margherita Pizza", category: "Pizza", price: 250, emoji: "[emoji]", desc: "Classic cheese and tomato
 pizza." },
  { id: 2, name: "Veg Burger", category: "Burgers", price: 120, emoji: "[emoji]", desc: "Crispy veg patty with fresh v
eggies." },
  { id: 3, name: "Chicken Biryani", category: "Rice", price: 220, emoji: "[emoji]", desc: "Aromatic basmati rice with
chicken." },
  { id: 4, name: "Masala Dosa", category: "South Indian", price: 90, emoji: "[emoji]", desc: "Crispy dosa with potato
masala." },
  { id: 5, name: "French Fries", category: "Snacks", price: 80, emoji: "[emoji]", desc: "Golden salted potato fries."
},
  { id: 6, name: "Chocolate Shake", category: "Drinks", price: 110, emoji: "[emoji]", desc: "Thick creamy chocolate mi
lkshake." },
  { id: 7, name: "Paneer Roll", category: "Snacks", price: 100, emoji: "[emoji]", desc: "Spicy paneer wrapped in a sof
t roll." },
  { id: 8, name: "Ice Cream Sundae", category: "Desserts", price: 130, emoji: "[emoji]", desc: "Vanilla ice cream with
 chocolate syrup." }
];

let cart = [];
try { cart = JSON.parse(localStorage.getItem("foodCart")) || []; } catch (e) { cart = []; }

const foodList = document.getElementById("foodList");
const searchInput = document.getElementById("searchInput");
const categoryFilter = document.getElementById("categoryFilter");
const cartItems = document.getElementById("cartItems");
const cartTotal = document.getElementById("cartTotal");
const cartCount = document.getElementById("cartCount");
const cartMessage = document.getElementById("cartMessage");
const formError = document.getElementById("formError");

function saveCart() { localStorage.setItem("foodCart", JSON.stringify(cart)); }

function loadCategories() {
  [...new Set(foods.map(f => f.category))].forEach(c => {
    const o = document.createElement("option");
    o.value = c; o.textContent = c;
    categoryFilter.appendChild(o);
  });
}

function renderFoods() {
  const term = searchInput.value.trim().toLowerCase();
  const cat = categoryFilter.value;
  const list = foods.filter(f =>
    (cat === "All" || f.category === cat) && f.name.toLowerCase().includes(term));
  foodList.innerHTML = list.length ? "" : "<p>No food items found.</p>";
  list.forEach(f => {
```

```
    const d = document.createElement("div");
    d.className = "card";
    d.innerHTML = `<div class="img">${f.emoji}</div>
      <div class="body"><h4>${f.name}</h4><span class="tag">${f.category}</span>
      <p>${f.desc}</p><div class="price">₹${f.price}</div>
      <button onclick="addToCart(${f.id})">Add to Cart</button></div>`;
    foodList.appendChild(d);
  });
}

function addToCart(id) {
  const item = cart.find(c => c.id === id);
  if (item) item.qty++; else cart.push({ id, qty: 1 });
  cartMessage.textContent = "";
  saveCart(); renderCart();
}

function changeQty(id, delta) {
  const item = cart.find(c => c.id === id);
  if (!item) return;
  item.qty += delta;
  if (item.qty <= 0) cart = cart.filter(c => c.id !== id);
  saveCart(); renderCart();
}

function removeItem(id) {
  cart = cart.filter(c => c.id !== id);
  saveCart(); renderCart();
}

function renderCart() {
  cartItems.innerHTML = "";
  let total = 0, count = 0;
  if (cart.length === 0) {
    cartItems.innerHTML = "<p>Your cart is empty.</p>";
  }
  cart.forEach(c => {
    const f = foods.find(x => x.id === c.id);
    const sub = f.price * c.qty;
    total += sub; count += c.qty;
    const row = document.createElement("div");
    row.className = "cart-row";
    row.innerHTML = `<span>${f.name}<br><small>₹${f.price} x ${c.qty} = ₹${sub}</small></span>
      <span class="qty"><button onclick="changeQty(${f.id},-1)">-</button>${c.qty}
      <button onclick="changeQty(${f.id},1)">+</button>
      <button onclick="removeItem(${f.id})">x</button></span>`;
    cartItems.appendChild(row);
  });
  cartTotal.textContent = total;
  cartCount.textContent = count;
}

function validateForm() {
  const name = document.getElementById("custName").value.trim();
  const phone = document.getElementById("custPhone").value.trim();
  const addr = document.getElementById("custAddress").value.trim();
  if (cart.length === 0) return "Cart is empty. Add items first.";
  if (name.length < 3) return "Enter a valid name (min 3 characters).";
  if (!/^[6-9]\d{9}$/.test(phone)) return "Enter a valid 10-digit phone number.";
  if (addr.length < 10) return "Enter a complete delivery address.";
  return "";
}

document.getElementById("reviewBtn").onclick = () => {
  const err = validateForm();
  formError.textContent = err;
  if (err) return;
  let html = "<table width='100%'>", total = 0;
  cart.forEach(c => {
    const f = foods.find(x => x.id === c.id);
    total += f.price * c.qty;
    html += `<tr><td>${f.name}</td><td>x${c.qty}</td><td>₹${f.price * c.qty}</td></tr>`;
  });
  html += `</table><h3>Total: ₹${total}</h3>
    <p><b>${document.getElementById("custName").value}</b><br>
    ${document.getElementById("custPhone").value}<br>
    ${document.getElementById("custAddress").value}</p>`;
  document.getElementById("summaryContent").innerHTML = html;
  document.getElementById("summaryModal").classList.remove("hidden");
};
```

```
document.getElementById("cancelBtn").onclick = () =>
  document.getElementById("summaryModal").classList.add("hidden");

document.getElementById("confirmBtn").onclick = () => {
  document.getElementById("summaryModal").classList.add("hidden");
  cart = []; saveCart(); renderCart();
  ["custName", "custPhone", "custAddress"].forEach(i => document.getElementById(i).value = "");
  cartMessage.textContent = "Order submitted successfully! Thank you.";
};

searchInput.addEventListener("input", renderFoods);
categoryFilter.addEventListener("change", renderFoods);

loadCategories(); renderFoods(); renderCart();
```

4.3 Project Execution

- Save the three files (index.html, style.css, script.js) in one folder.
- Double-click index.html (or use VS Code 'Live Server') to open it in a browser.
- Browse the menu, search by name or choose a category.
- Click 'Add to Cart', adjust quantities with + / -, and watch the total update.
- Fill name, phone and address, then click 'Review Order'.
- Check the order summary and click 'Confirm Order' to see the success message.

5. Output Screenshots

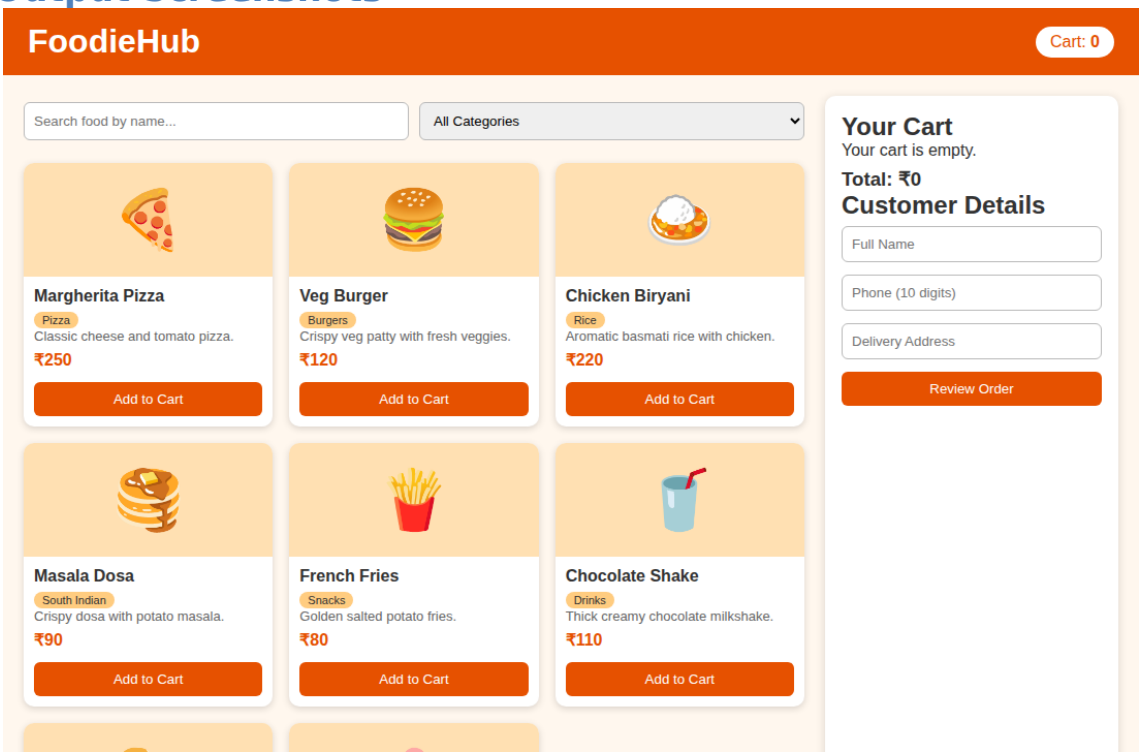


Fig 1: Home page with food items

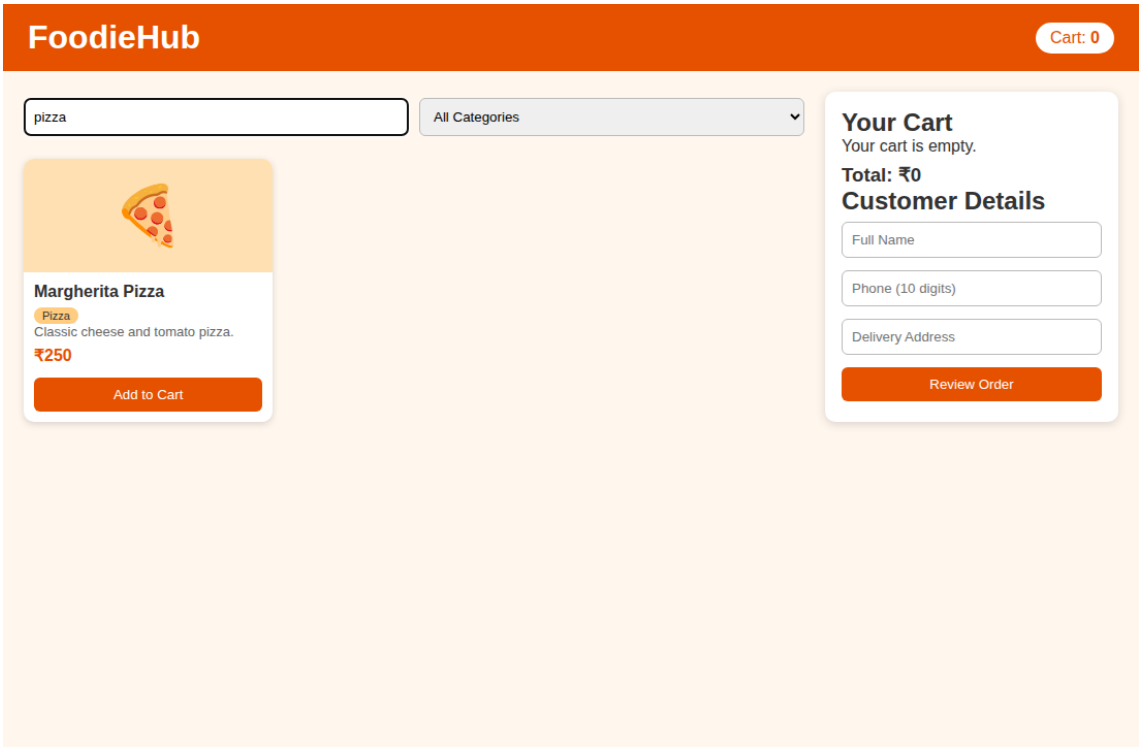


Fig 2: Search by name ("pizza")

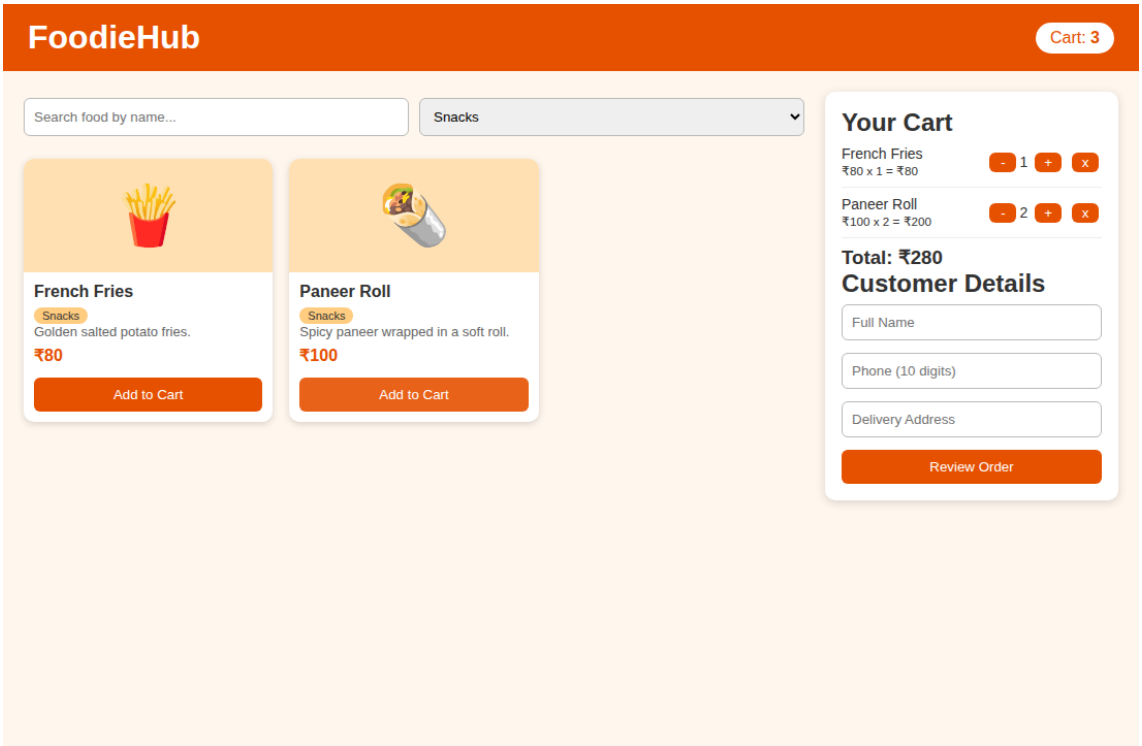


Fig 3: Category filter (Snacks) and cart with quantities and total

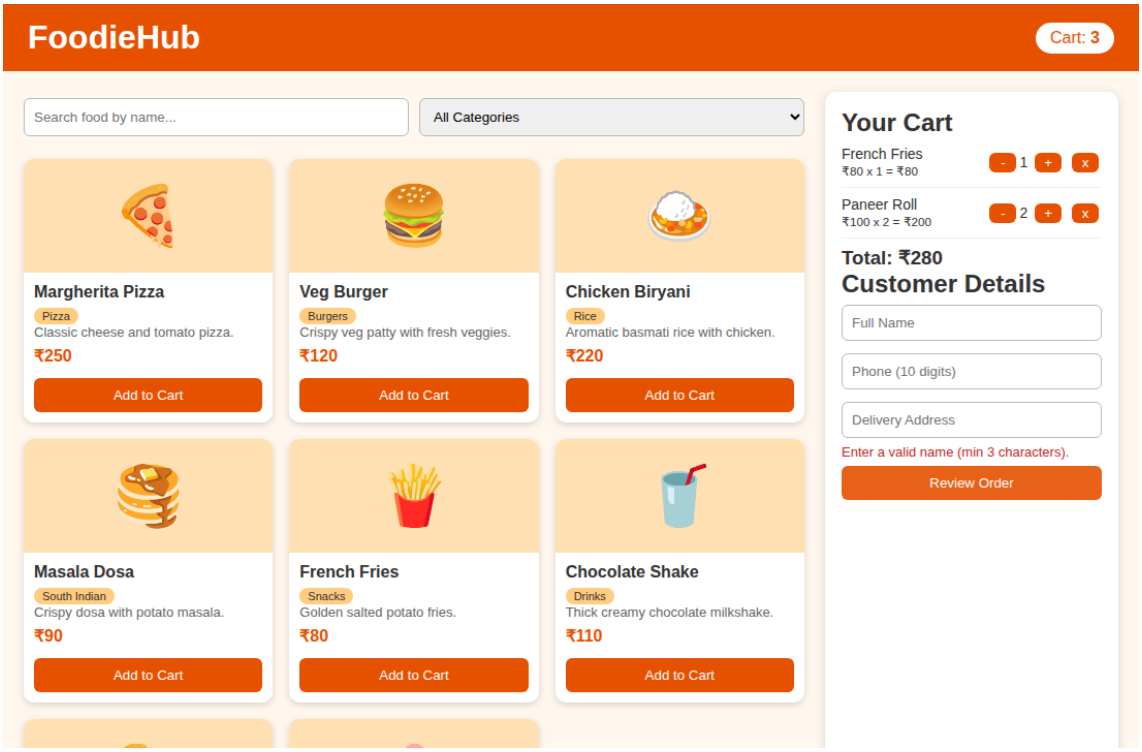


Fig 4: Validation message for missing customer details

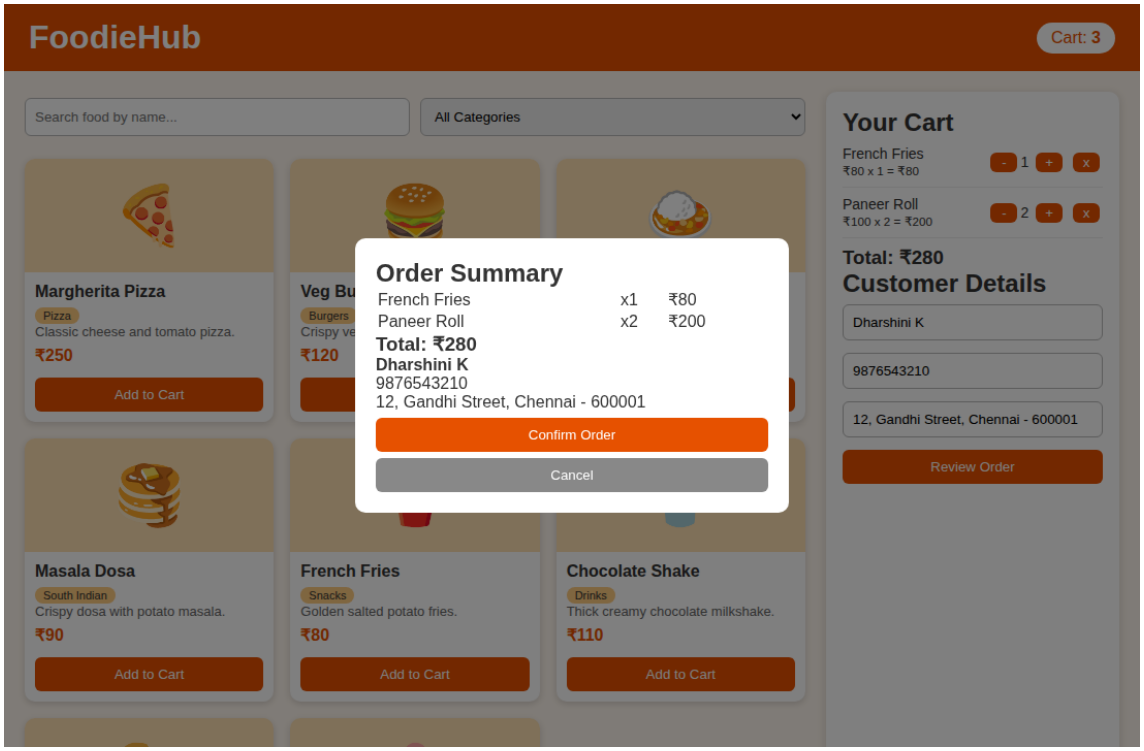


Fig 5: Order summary pop-up

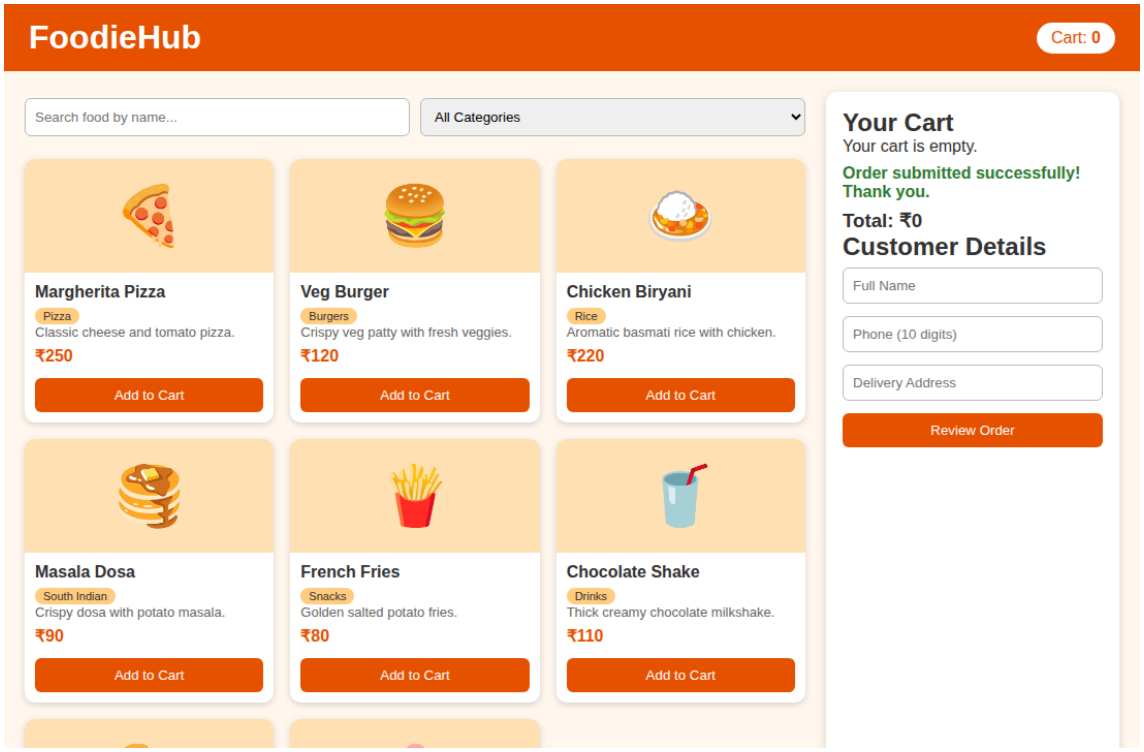


Fig 6: Order submitted message and empty cart

6. Technical Explanation

- **DOM manipulation:** createElement and innerHTML generate food cards and cart rows at run time.
- **Array methods:** filter() for search/category, find() to locate items, forEach() to render, and Set to build the category list.
- **Event handling:** 'input', 'change' and 'click' events make the page interactive without reloading.

- **Local Storage:** JSON.stringify() saves the cart and JSON.parse() restores it inside try/catch.
- **Validation logic:** a regular expression /^[6-9]\d{9}$/ checks the phone number; length checks validate name and address.
- **Responsive design:** Flexbox, Grid auto-fill and media queries adapt the layout to mobile and desktop.

7. Conclusion

The Online Food Ordering Website was successfully developed using HTML, CSS and JavaScript. It lets customers browse, search and filter dishes, manage a cart with quantities, see totals, validate details and review the order before submission. The cart persists through Local Storage. Through this project I learned DOM manipulation, event handling, array methods, form validation, Local Storage and responsive design.

8. Future Enhancements

- Add real food images and more menu items loaded from a backend database.
- User login/registration and order history.
- Online payment gateway integration (UPI, cards).
- Enhanced security: server-side validation and data encryption.
- Order tracking, ratings/reviews, discount coupons.
- Improved user experience with dark mode and multi-language support.